

Premiers pas en Python: Exercices complémentaires gradués

Etienne Dagorn*

Table des matières

1 Exercices Complémentaires	2
-----------------------------	---

*etienne.dagorn@univ-lille.fr

1 Exercices Complémentaires

Mode d'emploi pédagogique

Cette banque sert à consolider les TPs, pas à remplacer la progression principale. Les niveaux indiquent le degré d'autonomie attendu.

- **Niveau 1 – Lecture** : comprendre un code existant et prédire son résultat.
- **Niveau 2 – Modification** : changer légèrement un programme fourni.
- **Niveau 3 – Complétion** : compléter quelques lignes manquantes.
- **Niveau 4 – Construction** : écrire une solution complète à partir d'un objectif.
- **Niveau 5 – Transfert** : combiner plusieurs notions dans une situation nouvelle.

Commencer par un exercice de niveau 1 ou 2 lorsqu'une notion est encore récente, puis monter progressivement.

Lire un dictionnaire – Niveau 1

Avant d'écrire du code, prédire ce que ce programme affiche.

```
prix = {"pain": 1.2, "lait": 0.95}
print(prix["pain"])
print("pates" in prix)
print(prix.get("pates", 0))
```

Aide

Lire une clé existante avec `prix["pain"]`. Tester l'existence avec `in`. Utiliser `get(..., 0)` pour obtenir une valeur par défaut.

Modifier une boucle – Niveau 2

Le programme affiche les valeurs positives. Le modifier pour afficher seulement les valeurs supérieures ou égales à 10.

```
valeurs = [4, 10, 15, 2, 20]
for x in valeurs:
    if x > 0:
        print(x)
```

Aide

Une seule ligne est à modifier : la condition après `if`.

Débuguer un compteur – Niveau 3

Ce programme devrait compter les valeurs strictement positives, mais il ne s'exécute pas correctement. Identifier l'erreur, puis corriger.

```
valeurs = [4, -1, 0, 8]
for x in valeurs:
    if x > 0:
        compteur = compteur + 1
print(compteur)
```

Aide

Chercher quelle variable est utilisée avant d'avoir reçu une première valeur.

Fusion de dictionnaires – Niveau 4

Deux paniers contiennent des quantités :

```
p1 = {"pomme": 2, "pain": 1, "riz": 1}
p2 = {"pomme": 1, "yaourt": 4, "riz": 2}
```

- Fusionne p1 et p2 en p_total en **additionnant** les quantités lorsqu'une clé existe dans les deux.
- Trie p_total par quantité décroissante (affichage).

Aide

Crée un nouveau dict, copie p1, puis parcours p2 : si la clé existe déjà, additionne.
Tri d'affichage : `sorted(p_total.items(), key=lambda kv: -kv[1])`.

Bonus – Aide

Dans `fusion(p,q, mode)`, gère trois cas : "somme" (addition), "droite" (remplace par q), "gauche" (garde p).

Fréquence des lettres – Niveau 4

Demande un mot à l'utilisateur et affiche combien de fois chaque lettre apparaît (ex : banana ⇒ b:1, a:3, n:2).

- Affiche les lettres triées par fréquence décroissante.

Aide

Parcours caractère par caractère, garde seulement les lettres (`.isalpha()`), normalise en minuscules.

Compteur : `d[c] = d.get(c, 0) + 1`. Tri : `sorted(d.items(), key=lambda kv: -kv[1])`.

Bonus – Aide

Ignore les caractères non alphabétiques avant le comptage; filtre a-z via `.isalpha()` et `.lower()`.

Classement de notes – Niveau 3

À partir d'un dictionnaire {"Alice": 14, "Bob": 10, "Clara": 16}, affiche :

- Les noms triés par note décroissante.
- Le/la premier.ère et le/la dernier.ère.

Aide

Trie les `items()` du dictionnaire par la *valeur* décroissante : `key=lambda kv: -kv[1]`.
Le premier et le dernier sont accessibles par indices `[0]` et `[-1]`.

Bonus – Aide

Écris `top_k(d, k)` qui trie puis renvoie les `k` premiers éléments via un *slicing* `[:k]`.

Inventaire simple (prix & stock) – Niveau 5

On représente un inventaire par :

```
inventaire = {  
    "pomme": {"prix": 0.5, "stock": 20},  
    "pain":  {"prix": 1.2, "stock": 10},  
    "riz":   {"prix": 2.0, "stock": 5}  
}
```

- Calcule la valeur totale de l'inventaire (`prix * stock`, somme).
- Vends une commande {"pomme": 3, "riz": 2} si le stock est suffisant ; mets à jour les stocks.
- Ajoute une remise de 10% sur "riz" et recalcule la valeur totale.

Aide

Somme totale : parcours `inventaire.items()` et ajoute `prix * stock`.
Avant la vente : vérifie que chaque `stock` est suffisant, puis décrémente.
Remise : multiplie le `prix` par `0.9`, puis recalcule la somme.

Bonus – Aide

Pour trier par *valeur de stock* (`prix * stock`), trie `inventaire.items()` avec `key=lambda kv: -(kv[1]["prix"]*kv[1]["stock"])`.

Petites "stats" sur transactions – Niveau 5

On dispose d'une liste de transactions :

```
tx = [  
    {"nom": "Alice", "montant": 12.5},  
    {"nom": "Bob", "montant": 7.0},  
    {"nom": "Alice", "montant": 3.5},  
    {"nom": "Clara", "montant": 20.0}  
]
```

- Calcule la dépense totale par personne (dictionnaire `nom` $\rightarrow$ `total`).
- Qui a le plus dépensé ? Affiche le classement.

Aide

Cumule dans un dict : `totaux[nom] = totaux.get(nom,0) + montant`.
Classement : `sorted(totaux.items(), key=lambda kv: -kv[1])`.

Bonus – Aide

Stocke pour chaque nom `nb` d'achats et `total` ; la moyenne est `total/nb`.

Fonctions utilitaires (liste de nombres) – Niveau 4

Écris des fonctions :

- `moyenne(L)`, `mediane(L)` (liste triée/tri à faire), `mode(L)` (valeur la plus fréquente).
- Teste sur plusieurs listes (`[1,2,2,3]`, `[10]`, `[]`).

Aide

Moyenne : `sum/len` avec test liste vide.

Médiane : trie une *copie* ; pair $\Rightarrow$ moyenne des deux centraux.

Mode : dict de fréquences, `max()` sur les valeurs ; si ex aequo, renvoyer une liste.

Bonus – Aide

Pense à tester les cas limites : liste vide, un seul élément, multimodal.

Grouperments simples (sans pandas) – Niveau 5

On dispose d'un catalogue [`catégorie`, `produit`, `prix`] (liste de dicts).

- Calcule, par **catégorie**, le **nombre** de produits et le **prix moyen**.
- Affiche les catégories triées par prix moyen décroissant.

Aide

Crée un dict `groupes[cat] = {nb, somme}` en parcourant le catalogue.

Puis calcule `prix_moy = somme/nb` et regroupe dans une liste de tuples à trier.

Nettoyage léger – Niveau 3

À partir d'une liste de prénoms [`"alice "`, `"Bob"`, `"ALICE"`, `"bob "`, `" clara"`] :

- Supprime espaces superflus, harmonise en **Nom propre** (majuscule initiale seulement).
- Produit une liste **unique** triée.

Aide

Écris une petite fonction de nettoyage : `strip()`, `lower()`, `capitalize()`.

Construis la liste *unique* en testant `if x not in ...` puis trie avec `sorted()`.

Sauvegarder les résultats dans un fichier (100% Python) – Niveau 5

Dans cet exercice, vous allez apprendre à **enregistrer automatiquement les résultats de vos exercices précédents dans un fichier texte** en utilisant uniquement Python, sans passer par le terminal.

Avant de commencer

Vérifiez que vous savez déjà créer un fichier `.py`, l'enregistrer, l'exécuter et retrouver le fichier créé dans le dossier courant. Cet exercice introduit `open()`, qui est plus avancé que les sorties avec `print()`.

- Créez un nouveau script nommé `tp2_solutions.py`.
- En haut du script, importez le module `datetime` et ouvrez un fichier texte pour y écrire les résultats :

```
from datetime import datetime

# Creation (ou ecrasement) du fichier de resultats
fichier_resultats = open("resultats_tp2.txt", "w", encoding="utf-8")

# Fonction utilitaire : ecrit a l'ecran et dans le fichier
def log(texte):
    print(texte)
    fichier_resultats.write(str(texte) + "\n")

log("=== Resume des exercices du TP2 ===")
log("Execute le : " + str(datetime.now()))
log("") # ligne vide
```

- Pour chaque exercice du TP2 (1 à 15), ajoutez au moins un exemple dont le résultat sera enregistré dans le fichier en utilisant `log()` :

```
# Exemple : Exercice 2
notes = [12, 14, 10, 8]
moyenne = sum(notes) / len(notes)
log("Ex2 - Moyenne des notes : " + str(round(moyenne, 1)))
```

- Une fois tous les résultats enregistrés, n'oubliez pas de fermer le fichier pour le sauvegarder :

```
fichier_resultats.close()
```

À retenir

Après exécution de `tp2_solutions.py`, un fichier `resultats_tp2.txt` sera créé automatiquement dans le même dossier. Il contiendra toutes les sorties principales de vos exercices. Vous n'avez pas besoin d'utiliser le terminal : tout se fait directement en Python.